

Legal Document Intelligence

Technical Documentation & Deployment Plan

AI-assisted contract review, eDiscovery classification, and court docket monitoring for legal teams

Generated 2026-08-25
Internal / Confidential — for planning and deployment reference

1. Executive Summary

Legal Document Intelligence is a human-in-the-loop AI platform that helps corporate counsel, paralegals, litigation support teams, and compliance departments accelerate three workflows: contract clause review, eDiscovery document triage, and court docket monitoring. It ingests contracts (PDF/DOCX), extracts and risk-flags key clauses, classifies documents by type, tracks federal court dockets for new activity, and organizes all of this under case "matters" with role-based access control and an audit trail.

Every AI output — clause extraction, risk flags, document classification — is a first-pass suggestion for attorney review, never an automated decision. This is reflected throughout the UI ("Reviewed" checkboxes, confidence bands, explicit disclaimers) and in exported reports.

Current status: Build-order steps 1 through 8 are complete and tested (118 backend tests passing, frontend type-checks and builds cleanly). Steps 9-10 — formal legal-domain QA/UAT and production deployment — are the remaining work, covered in Sections 7-9 below.

2. Technology Stack

Layer	Technology	Notes
Backend framework	Python 3.10+, FastAPI, Uvicorn	Thin HTTP layer over an importable core library
Frontend	React 18 + TypeScript, Vite, react-router-dom v7	CSS Modules, no state library, no UI kit
Database	SQLite (stdlib sqlite3, no ORM)	Single file, shared by matters/dockets/auth; PostgreSQL is a future option if write concurrency ever becomes a bottleneck
NLP / ML	Hugging Face transformers + datasets, PyTorch	CPU inference only — see Section 4
Document parsing	pdfplumber (PDF), python-docx (Word)	
PDF report generation	reportlab (platypus)	Per-matter downloadable report
Docket monitoring	CourtListener / RECAP REST API via httpx	Diff-based change detection, not ML
Auth	PyJWT (HS256), bcrypt	Stateless bearer tokens, 8h expiry
Testing	pytest (backend), tsc + vite build (frontend)	118 backend tests, httpx.MockTransport for external API

Full pinned dependency list: `requirements.txt` (fastapi, uvicorn[standard], pdfplumber, python-docx, pydantic, pydantic-settings, python-multipart, transformers, httpx, reportlab, bcrypt, PyJWT) plus torch installed separately from the CPU-only PyTorch index — deliberately, to avoid pulling in CUDA packages this project never needs in either dev or production.

3. System Architecture

3.1 Layering

Two layers on the backend: **app/** is a thin FastAPI layer (routing, request/response schemas, auth dependencies) and **src/legalintel/** is the importable core library that does the actual work (parsing, extraction, risk-flagging, classification, docket monitoring, persistence). Routes call into the library; the library has no FastAPI dependency of its own, so it can be tested and reused independently.

3.2 Backend modules

- **app/api/routes/** — documents.py (parse / extract-clauses / classify), dockets.py (track / check / list / alerts / entries), matters.py (matter CRUD, per-matter clause-review persistence, PDF report), auth.py (login / logout / me / create-user / audit-log).
- **src/legalintel/ingestion/** — pdf_parser.py + docx_parser.py, dispatched by pipeline.py.
- **src/legalintel/extraction/** — clause_extractor.py: loads the fine-tuned QA model as a process-wide singleton, runs sliding-window extractive QA per clause category.
- **src/legalintel/risk/** — flagging.py: pure rule-based severity mapping and confidence banding on top of extraction output (no model).
- **src/legalintel/classification/** — document_classifier.py: 3-class sequence classification, returns the full softmax distribution for transparency.
- **src/legalintel/docket/** — courtlistener_client.py (rate-limit-aware REST client) and monitor.py (diff-based new-entry detection), no ML.
- **src/legalintel/matters/, auth/, reporting/** — persistence and PDF generation.
- **src/legalintel/storage.py** — single source of schema truth for every SQLite table, shared connect() helper used by every db.py module.

3.3 Frontend

Vite + React + TypeScript SPA. Routes: /login, / (matters list), /matters/:id (review interface — upload, browse persisted results, track dockets, download report, delete matter), /quick-analyze (ephemeral one-off analysis), /admin (attorney-only user management + audit log). All routes except /login require an active session via a RequireAuth wrapper. Talks to the API over CORS with a bearer token attached to every request; a 401 from any call clears the session immediately.

4. Machine Learning Models

Both models fine-tune the same small base checkpoint — **distilbert-base-uncased** (~66M parameters, ~265MB per fine-tuned checkpoint) — chosen specifically because it is light enough to run **inference on CPU** on ordinary hardware. Training happens once, on Google Colab's free GPU tier; the resulting checkpoint is downloaded and used locally/in production for inference only. No GPU is required anywhere in production.

4.1 Clause Extraction Model

Property	Value
Task framing	Extractive question-answering (CUAD/SQuAD2.0-style) — one fixed question per clause category, model points to the answer span in the contract text
Base checkpoint	distilbert-base-uncased
Architecture	AutoModelForQuestionAnswering
Training data	CUAD (Contract Understanding Atticus Dataset) — 510 contracts, 13,000+ expert annotations, filtered to 4 categories
Categories (of CUAD's 41)	Governing Law, Termination for Convenience, Uncapped Liability, Non-Compete
Tokenization	max_length=384, stride=128 (sliding window over long contracts)
Training config	learning_rate=3e-5, batch_size=8, epochs=3, weight_decay=0.01, fp16 on GPU, best checkpoint kept by eval loss
Inference	Sliding-window decoding per window + null-vs-span score comparison, so the model can report "no clause of this type found" instead of forcing a low-quality match
Output	Clause text span + confidence score + risk level (rule-based) + confidence band (HIGH/MEDIUM/LOW)
Trained where	Google Colab (notebooks/02_baseline_clause_extraction_colab.ipynb), free T4 GPU
Deployed as	models/clause-extraction-baseline/ — required for POST /documents/extract-clauses; endpoint fails fast with a clear error if missing

4.2 Document Classification Model

Property	Value
Task framing	3-class sequence classification
Base checkpoint	distilbert-base-uncased
Architecture	AutoModelForSequenceClassification (num_labels=3)
Classes	Contract, Email, Other
Tokenization	max_length=512
Training config	learning_rate=3e-5, batch_size=8, epochs=3, weight_decay=0.01, metric_for_best_model=f1

Property	Value
Output	Predicted label + full softmax probability distribution (not just the top label) for human-in-the-loop transparency
Trained where	Google Colab (notebooks/03_document_classification_colab.ipynb), free T4 GPU
Deployed as	models/document-classification-baseline/ — required for POST /documents/classify

Caveat — "Other" class: no clean public dataset of generic business memos, letters, or invoices exists, so "Other" is trained on generic news-article text as an explicit placeholder. This teaches the model "this is neither a contract nor an email" — it does **not** mean the model recognizes memos, invoices, or any other real eDiscovery document type. Treat predictions of "Other" as a triage signal, not a validated category, until a real proxy dataset replaces it.

4.3 Risk flagging (not a model)

Risk severity per clause category and confidence banding are pure rule-based Python (src/legalintel/risk/flagging.py) — a static severity table plus a confidence-to-band threshold function, run after extraction. No ML is involved in risk scoring itself.

5. Security & Access Control

Every API route except GET /health requires a bearer token. Authentication is stateless JWT (PyJWT, HS256, 8-hour expiry, subject = user id only, no refresh token) — chosen over cookies for a clean fit with the SPA + CORS architecture. Passwords are hashed with bcrypt. There is no public self-registration endpoint; the first account is bootstrapped via a CLI script (scripts/create_admin.py) run directly against the database by a trusted operator.

get_current_user re-checks the user's is_active flag against the database on **every request** rather than trusting a role embedded in the token, so deactivating a user takes effect immediately with no token blacklist required.

5.1 Roles & permissions

Action	Attorney	Paralegal	Support Staff
View matters, documents, dockets, own audit visibility	Yes	Yes	Yes (read-only)
Create matters; upload/analyze documents; track dockets; mark clauses reviewed	Yes	Yes	No
Delete a matter	Yes	No	No
Create users; view the full audit log	Yes	No	No

5.2 Audit trail

Logged events: login, logout, matter creation, matter deletion, and clause-review actions (who reviewed which clause, when — an upsert, so re-review overwrites the current reviewer while the audit_log entry preserves history). Document analysis, docket checks, and report downloads are **not** individually logged — a deliberate scope decision, not an oversight.

5.3 Deferred: encryption at rest

Documents and analysis results are not currently encrypted at rest in the SQLite database. This is a tracked, deliberate follow-up rather than an oversight — flagged here for the deployment decision in Section 8 if this system will hold real client data in production.

6. Data Model

Single SQLite database, schema owned entirely by src/legalintel/storage.py:

Table	Purpose
users	Accounts: email, name, password hash, role, active flag
matters	Case/project container; created_by references users
matter_documents	One row per persisted analysis (parse/extract_clauses/classify), opaque result_json blob
tracked_dockets	CourtListener dockets being monitored, optionally linked to a matter

Table	Purpose
seen_docket_entries	Dedup state for docket change-detection
docket_alerts	Alert history when new docket entries are found
audit_log	Who did what, when (see 5.2)
clause_reviews	Current review state per (matter_document, clause_index) — upsert, not a log

7. Development Plan (Build Order)

#	Step	Status
1	Document ingestion pipeline (PDF/DOCX parsing, text extraction)	Done
2	Baseline clause-extraction model (Colab, extractive QA)	Done
3	Risk-flagging logic + confidence scoring	Done
4	eDiscovery document classification model (Colab)	Done
5	Docket monitoring integration + change detection + alerting	Done
6	Review interface — highlighted clauses, risk scores, matter organization	Done
7	Reporting / export for stakeholders (PDF)	Done
8	Role-based access control + audit trail	Done
9	Testing & QA against legal-domain review	Remaining
10	UAT, refinement, deployment	Remaining — Sections 8-9 below

Additionally completed outside the numbered build order: a premium dark-themed UI redesign across the whole frontend.

7.1 Test coverage

- 118 backend tests passing (pytest) — unit tests per module plus route-level TestClient tests with real auth fixtures for every protected endpoint.
- Model-backed test packages (extraction, classification) use a throwaway untrained-head model built from the base checkpoint, so tests exercise the code path without needing the real trained weights or asserting on prediction quality.
- Docket tests never hit the real CourtListener API — httpx.MockTransport doubles throughout.
- Frontend: tsc type-checks clean and vite build succeeds with zero errors.

8. Recommended Deployment Architecture

The current architecture (FastAPI + SQLite + static React build) is intentionally simple and does not require a GPU, a managed database, or a container orchestration platform to run in production at small-to-medium law-firm scale. The recommended baseline is a single virtual private server:

Component	Recommendation
Compute	1 Linux VPS — Ubuntu 22.04 LTS, 2 vCPU / 4GB RAM / 80GB SSD for a pilot; 4 vCPU / 8GB RAM for moderate concurrent usage
Backend process	Uvicorn workers behind Nginx as a reverse proxy, run as a systemd service (not --reload)

Component	Recommendation
Frontend	Static React build (npm run build) served by the same Nginx, or a static host (e.g. Netlify/Vercel free tier) pointed at the API via VITE_API_BASE_URL
Database	SQLite file on local SSD — matches the current architecture; revisit only if concurrent-write contention actually appears
TLS	Let's Encrypt via Certbot, auto-renewing — free
Model checkpoints	~530MB combined (both models) copied to the server out-of-band (scp/rsync) — not committed to git
Backups	Nightly SQLite file copy + off-site upload (S3/Backblaze B2); provider snapshot as a second layer
Firewall	ufw allowing only 22 (SSH), 80, 443

8.1 Deployment procedure

- 1. Provision the VPS (Ubuntu 22.04 LTS) and point a domain's A record at it.
- 2. Install system packages: Python 3.10+, Node.js LTS (for the frontend build), Nginx, Certbot, git.
- 3. Clone the repository; create a venv; `pip install torch --index-url https://download.pytorch.org/whl/cpu` then `pip install -r requirements.txt`.
- 4. Copy the two trained model folders (clause-extraction-baseline/, document-classification-baseline/) onto the server via scp/rsync — do not commit these to git.
- 5. Create .env with JWT_SECRET_KEY (a fresh random 32-byte secret — never reuse the dev value), COURTLISTENER_API_TOKEN, DB path, CORS allowed origins, and upload limits.
- 6. Bootstrap the first attorney account: `python -m scripts.create_admin --email you@firm.com --name "..."`.
- 7. Run the backend as a systemd service (Uvicorn with multiple workers, or Uvicorn behind Gunicorn) instead of the dev --reload flag.
- 8. Build the frontend (npm run build) and serve the static output via Nginx; configure Nginx to reverse-proxy /api-prefixed (or all) traffic to the Uvicorn backend.
- 9. Issue a TLS certificate with Certbot and confirm auto-renewal is scheduled.
- 10. Configure the firewall (ufw) and disable password SSH login in favor of keys.
- 11. Set up the nightly SQLite backup cron job and verify a restore actually works before going live.
- 12. Smoke-test the full flow in production: login, create a matter, analyze a document, track a docket, download a report, confirm role boundaries for a paralegal and support-staff account.

9. Deployment Cost Estimate

These are planning-level estimates based on typical public cloud pricing at the time of writing (2026). Actual pricing varies by provider, region, and current promotions — get a live quote from the chosen provider before committing budget. No GPU cost is included anywhere: both models run inference on CPU in production, which is the main cost lever this architecture already saves.

9.1 Pilot / low-traffic tier (single small firm, light concurrent use)

Item	Est. cost	Notes
VPS — 2 vCPU / 4GB RAM / 80GB SSD	\$18 - \$24 / mo	DigitalOcean/Linode/Vultr-class droplet
Automated VPS snapshots	\$3 - \$5 / mo	Provider-managed, ~20% of droplet cost
Off-site backup storage	\$0 - \$1 / mo	A few GB of SQLite backups on S3/Backblaze B2
Domain name	~\$1 / mo	\$10-15/year, amortized
TLS certificate	\$0	Let's Encrypt
CourtListener API	\$0	Free tier (5/min, 50/hr, 125/day) — sufficient for on-demand, non-polling use
Uptime monitoring	\$0	Free tier (e.g. UptimeRobot) is sufficient at this scale
Estimated total	\$22 - \$31 / mo	~\$265 - \$375 / year

9.2 Moderate-traffic tier (several concurrent users, more headroom)

Item	Est. cost	Notes
VPS — 4 vCPU / 8GB RAM / 160GB SSD	\$42 - \$48 / mo	
Automated VPS snapshots	\$8 - \$10 / mo	
Off-site backup storage	\$1 - \$2 / mo	
Domain name	~\$1 / mo	
Uptime monitoring (paid tier, optional)	\$0 - \$7 / mo	Optional; free tier still usually adequate
Estimated total	\$52 - \$68 / mo	~\$625 - \$815 / year

9.3 Not included in the estimates above

- Developer/DevOps time to execute the deployment procedure in Section 8.1 — engagement-dependent, not a recurring infrastructure cost.
- Managed PostgreSQL, if a future migration off SQLite is ever triggered by real write concurrency (roughly \$15-60/month depending on provider and tier) — not needed at this architecture's current scale.
- Encryption-at-rest tooling if implemented (Section 5.3) — typically low single-digit \$/month for a managed KMS, or \$0 if handled via disk-level encryption already offered by most VPS providers.

- A CourtListener paid tier, only if usage ever needs to exceed the free rate limits — CourtListener does not publish fixed paid pricing; contact them directly if this becomes necessary.
- Outbound email/SMS notifications, if alerting is extended beyond the in-app docket alert list — SendGrid/Twilio-class services have free tiers adequate for low volume.

9.4 Bottom line

A production-ready pilot deployment of this system, as architected today, costs roughly **\$22-31/month (~\$265-375/year)** in infrastructure, scaling to roughly **\$52-68/month (~\$625-815/year)** for moderate concurrent usage — plus one-time developer time to execute the deployment procedure. There is no licensing cost for any component in the stack (all open-source), and no GPU/inference-hosting cost since both models are small enough to run on ordinary CPU.

This document was generated to accompany the codebase as a planning and reference artifact. It summarizes the system as implemented at the time of generation and the deployment/cost figures are estimates for planning purposes, not a vendor quote or a guarantee of future pricing. Legal-domain QA (build-order step 9) has not yet been completed — do not treat model outputs as validated for production legal decision-making until that step is done.